

SUBQUERY ASSIGNMENT

SQL Questions Q151–Q200

```
-- =====
-- SECTION 4 – SUBQUERY
-- =====

-- Q151. Display products whose price is greater than the average product price.
SELECT product_name, price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
);

-- Q152. Display products whose price is less than the average product price.
SELECT product_name, price
FROM products
WHERE price < (
    SELECT AVG(price)
    FROM products
);

-- Q153. Display the product with the highest price.
SELECT product_name, price
FROM products
WHERE price = (
    SELECT MAX(price)
    FROM products
);

-- Q154. Display the product with the lowest price.
SELECT product_name, price
FROM products
WHERE price = (
    SELECT MIN(price)
    FROM products
);

-- Q155. Display products having stock greater than the average stock.
SELECT product_name, stock
FROM products
WHERE stock > (
    SELECT AVG(stock)
    FROM products
);

-- Q156. Display products having stock less than the average stock.
SELECT product_name, stock
FROM products
WHERE stock < (
    SELECT AVG(stock)
    FROM products
);

-- Q157. Display orders whose total amount is greater than the average order amount.
SELECT order_id, total_amount
FROM orders
WHERE total_amount > (
    SELECT AVG(total_amount)
    FROM orders
);

-- Q158. Display orders whose total amount is less than the average order amount.
SELECT order_id, total_amount
FROM orders
WHERE total_amount < (
    SELECT AVG(total_amount)
    FROM orders
);

-- Q159. Display the order with the highest total amount.
SELECT order_id, total_amount
FROM orders
```

```

WHERE total_amount = (
    SELECT MAX(total_amount)
    FROM orders
);

-- Q160. Display the order with the lowest total amount.
SELECT order_id, total_amount
FROM orders
WHERE total_amount = (
    SELECT MIN(total_amount)
    FROM orders
);

-- Q161. Display customers who have placed at least one order.
SELECT customer_id, customer_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
);

-- Q162. Display customers who have not placed any order.
SELECT customer_id, customer_name
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM orders
);

-- Q163. Display products that have been ordered at least once.
SELECT product_id, product_name
FROM products
WHERE product_id IN (
    SELECT product_id
    FROM order_items
);

-- Q164. Display products that have never been ordered.
SELECT product_id, product_name
FROM products
WHERE product_id NOT IN (
    SELECT product_id
    FROM order_items
);

-- Q165. Display orders that have a payment record.
SELECT order_id, total_amount
FROM orders
WHERE order_id IN (
    SELECT order_id
    FROM payments
);

-- Q166. Display orders that do not have a payment record.
SELECT order_id, total_amount
FROM orders
WHERE order_id NOT IN (
    SELECT order_id
    FROM payments
);

-- Q167. Display orders having a shipping record with a delivered status.
SELECT order_id, total_amount
FROM orders
WHERE shipping_id IN (
    SELECT shipping_id
    FROM shipping
    WHERE shipping_status = 'Delivered'
);

-- Q168. Display orders having a shipping record with a pending status.
SELECT order_id, total_amount
FROM orders
WHERE shipping_id IN (
    SELECT shipping_id
    FROM shipping

```

```

        WHERE shipping_status = 'Pending'
    );

-- Q169. Display products belonging to the category 'Laptops'.
SELECT product_name, price
FROM products
WHERE category_id = (
    SELECT category_id
    FROM categories
    WHERE category_name = 'Laptops'
);

-- Q170. Display products belonging to the category 'Mobile Phones'.
SELECT product_name, price
FROM products
WHERE category_id = (
    SELECT category_id
    FROM categories
    WHERE category_name = 'Mobile Phones'
);

-- Q171. Display products belonging to the brand 'Samsung'.
SELECT product_name, price
FROM products
WHERE brand_id = (
    SELECT brand_id
    FROM brands
    WHERE brand_name = 'Samsung'
);

-- Q172. Display products belonging to the brand 'Apple'.
SELECT product_name, price
FROM products
WHERE brand_id = (
    SELECT brand_id
    FROM brands
    WHERE brand_name = 'Apple'
);

-- Q173. Display customers who live in the same city as 'Aarav Sharma'.
SELECT customer_name, city
FROM customers
WHERE city = (
    SELECT city
    FROM customers
    WHERE customer_name = 'Aarav Sharma'
);

-- Q174. Display customers who signed up before 'Aarav Sharma'.
SELECT customer_name, signup_date
FROM customers
WHERE signup_date < (
    SELECT signup_date
    FROM customers
    WHERE customer_name = 'Aarav Sharma'
);

-- Q175. Display orders placed by 'Aarav Sharma'.
SELECT order_id, order_date, total_amount
FROM orders
WHERE customer_id = (
    SELECT customer_id
    FROM customers
    WHERE customer_name = 'Aarav Sharma'
);

-- Q176. Display products whose price is greater than the average price of products in category 1.
SELECT product_name, price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
    WHERE category_id = 1
);

-- Q177. Display products whose stock is greater than the average stock of products in category 1.

```

```

SELECT product_name, stock
FROM products
WHERE stock > (
    SELECT AVG(stock)
    FROM products
    WHERE category_id = 1
);

-- Q178. Display orders whose amount is greater than the average amount of orders placed by customer 1.
SELECT order_id, total_amount
FROM orders
WHERE total_amount > (
    SELECT AVG(total_amount)
    FROM orders
    WHERE customer_id = 1
);

-- Q179. Display customers who have placed more than one order.
SELECT customer_id, customer_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
    GROUP BY customer_id
    HAVING COUNT(*) > 1
);

-- Q180. Display products that have been ordered more than once.
SELECT product_id, product_name
FROM products
WHERE product_id IN (
    SELECT product_id
    FROM order_items
    GROUP BY product_id
    HAVING COUNT(*) > 1
);

-- Q181. Display customers whose total order amount is greater than 10000.
SELECT customer_id, customer_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
    GROUP BY customer_id
    HAVING SUM(total_amount) > 10000
);

-- Q182. Display products whose total ordered quantity is greater than 5.
SELECT product_id, product_name
FROM products
WHERE product_id IN (
    SELECT product_id
    FROM order_items
    GROUP BY product_id
    HAVING SUM(quantity) > 5
);

-- Q183. Display categories that contain at least one product.
SELECT category_id, category_name
FROM categories
WHERE category_id IN (
    SELECT category_id
    FROM products
);

-- Q184. Display brands that contain at least one product.
SELECT brand_id, brand_name
FROM brands
WHERE brand_id IN (
    SELECT brand_id
    FROM products
);

-- Q185. Display customers using EXISTS who have placed an order.
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (

```

```

SELECT 1
FROM orders o
WHERE o.customer_id = c.customer_id
);

-- Q186. Display customers using NOT EXISTS who have not placed an order.
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);

-- Q187. Display products using EXISTS that have been ordered.
SELECT p.product_id, p.product_name
FROM products p
WHERE EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);

-- Q188. Display products using NOT EXISTS that have never been ordered.
SELECT p.product_id, p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
);

-- Q189. Display orders using EXISTS that have a payment record.
SELECT o.order_id, o.total_amount
FROM orders o
WHERE EXISTS (
    SELECT 1
    FROM payments p
    WHERE p.order_id = o.order_id
);

-- Q190. Display orders using NOT EXISTS that do not have a payment record.
SELECT o.order_id, o.total_amount
FROM orders o
WHERE NOT EXISTS (
    SELECT 1
    FROM payments p
    WHERE p.order_id = o.order_id
);

-- Q191. Display products whose price is greater than the average price of their own category.
SELECT p.product_name, p.category_id, p.price
FROM products p
WHERE p.price > (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.category_id = p.category_id
);

-- Q192. Display products whose stock is greater than the average stock of their own category.
SELECT p.product_name, p.category_id, p.stock
FROM products p
WHERE p.stock > (
    SELECT AVG(p2.stock)
    FROM products p2
    WHERE p2.category_id = p.category_id
);

-- Q193. Display products whose price is lower than the average price of their own brand.
SELECT p.product_name, p.brand_id, p.price
FROM products p
WHERE p.price < (
    SELECT AVG(p2.price)
    FROM products p2
    WHERE p2.brand_id = p.brand_id
);

-- Q194. Display products whose stock is greater than the average stock of their own brand.

```

```

SELECT p.product_name, p.brand_id, p.stock
FROM products p
WHERE p.stock > (
    SELECT AVG(p2.stock)
    FROM products p2
    WHERE p2.brand_id = p.brand_id
);

```

```

-- Q195. Display orders whose amount is greater than the average order amount of the same customer.
SELECT o.order_id, o.customer_id, o.total_amount
FROM orders o
WHERE o.total_amount > (
    SELECT AVG(o2.total_amount)
    FROM orders o2
    WHERE o2.customer_id = o.customer_id
);

```

```

-- Q196. Display order items whose price is greater than the average price of the same product in order items.
SELECT oi.order_item_id, oi.product_id, oi.price
FROM order_items oi
WHERE oi.price > (
    SELECT AVG(oi2.price)
    FROM order_items oi2
    WHERE oi2.product_id = oi.product_id
);

```

```

-- Q197. Display customers who have at least one order above the overall average order amount.
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
    AND o.total_amount > (
        SELECT AVG(total_amount)
        FROM orders
    )
);

```

```

-- Q198. Display products that have at least one order item with quantity greater than the average quantity.
SELECT p.product_id, p.product_name
FROM products p
WHERE EXISTS (
    SELECT 1
    FROM order_items oi
    WHERE oi.product_id = p.product_id
    AND oi.quantity > (
        SELECT AVG(quantity)
        FROM order_items
    )
);

```

```

-- Q199. Display customers whose total order amount is greater than the average total order amount per customer.
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE c.customer_id IN (
    SELECT o.customer_id
    FROM orders o
    GROUP BY o.customer_id
    HAVING SUM(o.total_amount) > (
        SELECT AVG(customer_total)
        FROM (
            SELECT customer_id, SUM(total_amount) AS customer_total
            FROM orders
            GROUP BY customer_id
        ) AS customer_totals
    )
);

```

```

-- Q200. Display the customer name and total order amount using a subquery in the FROM clause.
SELECT customer_totals.customer_name,
       customer_totals.total_amount
FROM (
    SELECT c.customer_name,
           SUM(o.total_amount) AS total_amount
    FROM customers c
    INNER JOIN orders o
    ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.customer_name
)

```

```
) AS customer_totals;
```